

DIGITALES BUERO

Entwickler-Handbuch & Architektur-Referenz

Vollstaendige Dokumentation fuer weiterfuehrende Entwickler
Software-Architektur, Module, Datenmodelle & CI/CD

Version: 1.2.5 (Build 17) | Plattform: iOS 17.0+ | Language: Swift 5.9 / 6.0

Repository: DevDealer05/DokuSys | Branch: main / gh-pages (SideStore)

Status: CI/CD Build Passed - SideStore OTA Live

Inhaltsverzeichnis

1. Executive Summary & Design-System (Liquid Glass)	Seite 2
2. System-Architektur & Komponenten-Diagramm	Seite 3
3. Modul-Katalog: Alle Swift-Dateien im Detail	Seite 4
4. Dokumenten-Ersteller (DIN 5008) & KI-Assistent	Seite 5
5. CI/CD Build-Pipeline, SideStore & Lessons Learned	Seite 6

1. Executive Summary & Design-Philosophie

Projekt-Mission

Digitales Buero ist eine spezialisierte iOS-Anwendung fuer persoenliches Dokumenten- und Schuldenmanagement. Die Kernidee: Ein intelligenter KI-Buerobegleiter, der Fristen ueberwacht, Zahlungsaufforderungen via OCR scannt, unberechtigte Inkassogebeuehren abwehrt und formelle DIN-5008-Antwortschreiben mit digitaler Signatur in Sekunden erstellt.

Liquid Glass Design-System (Dark Mode)

Token / Modifier		
Theme.primaryAccent	#6E5BE8	Leuchtendes Indigo-Lila als Primaerfarbe
Theme.primaryGradient	#6E5BE8 -> #8E7CFF	Farbverlauf fuer primaere CTAs
Theme.appBackground	#0B0D17 -> #07080E	Tiefschwarzer Hintergrund-Gradient
Theme.glassEdgeGradient	White 0.25 -> 0.05	Duenne Lichtkante fuer Glass-Karten
.liquidGlassCard()	ViewModifier	Transluzente Karte mit Blur & Schatten

Kern-Frameworks & Schnittstellen

1. Apple VisionKit & Vision: Belegerkennung, Text-OCR und Metadaten-Extraktion.
2. Apple PencilKit: Digitale Unterschriftserfassung (PKCanvasView) im DIN 5008 Assistenten.
3. Apple PDFKit & UIKit: Praezise Vektor-PDF-Erstellung (UIGraphicsPDFRenderer).
4. Apple LocalAuthentication: Face ID / Touch ID App-Sperre mit PIN-Fallback.
5. Supabase Client (PostgreSQL 15, Auth, RLS, Storage Buckets, Deno Edge Functions).
6. Google Gemini 3.6 Flash / Pro via Server-Sent Events (SSE) Streaming.
7. Integrierter Diagnose-Webserver: RemoteLogServer auf Port 8080 fuer LAN-Debugging.

2. System-Architektur & Datenfluss

SCHICHTEN-ARCHITEKTUR & DATENFLUSS

Praesentations-Schicht (SwiftUI Views)

DocumentArchiveView, DocumentCreatorView, DebtDetailView, AIChatView

Service- & Logik-Schicht (@MainActor ObservableObjects)

DocumentArchiveService, DebtEngineService, GeminiService, RemoteLogServer

Persistenz- & Cloud-Schicht

Lokal: FileManager (/dms_files/<UUID>.pdf) + UserDefaults JSON Caches

Cloud: Supabase PostgreSQL (debts, documents) + Gemini 3.6 Flash LLM

Offline-First Paradigma & Synchronisation

Optimistische UI-Updates

1. Alle Schreiboperationen (Dokumente, Raten, Schuldenstatus) werden sofort lokal gespeichert und unmittelbar in den @Published Collections der Services reflektiert.
2. Die App funktioniert 100% autark ohne Internetverbindung.
3. PDF-Dateien werden direkt im App-Sandbox-Ordner dms_files/<UUID>.pdf persistiert.
4. Bei aktiver Netzwerkverbindung synchronisiert SupabaseConfig im Hintergrund mit der Cloud.
5. Fehlerhafte Remote-Requests blockieren niemals die UI oder das lokale Speichern.

Sicherheit & Face ID Schutz

- Biometrie-Sperre: FaceIDGuard sperrt das UI sofort bei Eintritt in den Hintergrund.
- Datenschutz: Keine sensiblen Schuldnerdaten im Plaintext in ungeschuetzten Logs.
- Zero-Hardcoded-Secrets: Alle Tokens & API-Keys ueber UserDefaults oder Env-Vars.

3. Modul-Katalog: Alle Swift-Dateien

DigitalesBueroApp.swift	Einstiegspunkt (@main), FloatingTabBarView, Face ID Guard
DocumentArchiveModule.swift	DMS mit Volltextsuche, Fristen-Radar, Quick-Chips & AppDocument
DocumentCreatorModule.swift	DIN 5008 4-Schritte-Assistent, 7 Vorlagen, Signatur & PDF-Ablage
DebtEngineService.swift	Schulden-Engine, Glaebiger, Aktenzeichen, Verjaehrung (Paragraph 195 BGB)
DebtDetailView.swift	Detailansicht fuer Schulden mit Timeline, Zinsen & Statusverwaltung
ScannerModule.swift	VisionKit Kamera-Scanner, Live-OCR, Triage & Belegextraktion
ExportModule.swift	Dossier- & Ratenzahlungs-PDF-Export mit Unterschrift & ShareSheet
HardwareLogModule.swift	Wartungsbuch fuer E-Scooter, Haushaltsgeraete mit Kosten-Tracking
RealtimeHouseholdService.swift	Haushalts-Ausgaben, Mitglieder-Verwaltung & Budget-Sharing
AIChatModule.swift	KI-Chat-Interface mit App-Assistent und Code-Aenderungs-Modus
GeminiService.swift	Gemini 3.6 SSE-Streaming, Modell-Fallback & Code-Agent Integration
DevModeModule.swift	Entwickler-Panel, Build-Info, SideStore-URL, Testdaten-Generator
RemoteLogServer.swift	Integrierter HTTP-Server (Port 8080) zur Live-Log-Inspektion im WLAN
AppLogger.swift	Zentraler Thread-sicherer Ringspeicher fuer Log-Nachrichten
LiquidGlass.swift	Design-System, Farben, Farbverlaeuft, .liquidGlassCard() Modifier
UserSessionManager.swift	Lokales Profil (Name, Anschrift, Budget) fuer Absenderzeilen
CommercialModule.swift	Lizenz-Verwaltung & In-App-Freischaltungen

4. Dokumenten-Ersteller & KI-Assistent

DER 4-SCHRITTE WIZARD (DocumentAssistantCreatorView)

Schritt 1: Zweck / Vorlagenauswahl

- Widerspruch Inkasso (Paragraph 4 Abs. 5 RDGEG, Paragraph 174 BGB Vollmachtsruege)
- Kuendigung Vertrag/Abo, Ratenzahlungsangebot, Fristverlaengerung
- DSGVO-Auskunft (Art. 15), Einrede der Verjaehrung (Paragraph 214 BGB), Freier KI-Brief

Schritt 2: Daten & Adressen - 100% editierbare Felder, Vorbefuellung aus UserProfile

Schritt 3: Text-Editor & KI-Feinschliff mit Tonalitaet (Rechtssicher/Kooperativ/Praegnant)

Schritt 4: DIN 5008 Briefvorschau, PencilKit Unterschrift & automatische DMS-Ablage

Autonomer Code-Mutation Workflow (KI-Agent)

Vom In-App Chat direkt zum Git-Commit

1. Nutzer waehlt im KI-Chat den Modus 'Code aendern'.
2. GeminiService sendet Prompt an Gemini 3.6 Flash mit vollem Architektur-Briefing.
3. KI generiert Antwort mit CODE_CHANGES_JSON:[{file: '...', content: '...'}] Tag.
4. GeminiService sendet Aenderungen an Supabase Edge Function ai-agent.
5. Edge Function committed via GitHub API auf den main Branch.
6. GitHub Actions startet automatisch Build & aktualisiert die SideStore-IPA!

Diagnose: Web-Logserver auf Port 8080

Zur Fehlersuche auf physischen Geraeten startet RemoteLogServer einen HTTP-Server.

Aufruf im Browser desselben Netzwerks: `http://<IP-des-iPhones>:8080`

Features der Diagnose-Oberflaeche:

- Farbkodierte Live-Logs (INFO, WARN, ERROR) aus dem Ringspeicher
- System-Informationen (Akkustand, Speicher, iOS-Version, Build-Nummer)
- Filter nach Modul (z. B. 'KI-Chat', 'DMS', 'Scanner')

5. CI/CD Build-Pipeline & Lessons Learned

GITHUB ACTIONS PIPELINE (.github/workflows/build_ipa.yml)

1. Checkout & Select Xcode 16 auf macos-14 Runner
2. Versionsauslese aus Package.swift (displayVersion: 1.2.5, bundleVersion: 17)
3. Icon-Generierung via scripts/generate_icons.sh
4. xcodebuild mit AD_HOC_CODE_SIGNING_ALLOWED=YES (Unsigned Bundle)
5. Packaging via scripts/package_ipa.sh -> DigitalesBuero.ipa
6. Generierung des SideStore Manifests -> apps.json
7. Automatisches Deployment auf gh-pages Branch via peaceiris/actions-gh-pages

Wichtige Compiler-Regeln & Lessons Learned

Kritische Compiler-Regeln fuer neue Entwickler:

REGEL 1: ShapeStyle Ternaer-Typen in SwiftUI

Niemals: isSelected ? Theme.primaryAccent : Theme.glassEdgeGradient schreiben.

Color und LinearGradient sind unterschiedliche Typen!

Richtig: isSelected ? AnyShapeStyle(Theme.primaryAccent) : AnyShapeStyle(Theme.glassEdgeGradient)

REGEL 2: CoreGraphics Dictionary Keys in UIKit

UIGraphicsPDFRendererFormat.documentInfo erwartet [String: Any].

kCGPDFContext-Keys sind CFString. Immer: kCGPDFContextCreator as String verwenden.

REGEL 3: AppDocument lokale Dateipfade

Immer relativen localFileName ('<UUID>.pdf') speichern, absolute Pfade zur Laufzeit ueber FileManager.default.urls(for: .documentDirectory, ...) aufloesen!

REGEL 4: Keine Secrets im Quellcode

API-Keys gehoeren in UserDefaults ('gemini_api_key') oder GitHub Repository Secrets.

SideStore Repository URL fuer das iPhone:

<https://DevDealer05.github.io/DokuSys/apps.json>